Aegivex AI - Database Design Document

Version: 1.0
Document Type: Database Design Document
Project: OKX.AI Genesis Hackathon
Prepared For: Development Team
Prepared By: Aegivex AI Team

1. Overview

This document defines the database architecture for **Aegivex AI**. The database is designed to store user information, scan history, AI conversations, security reports, and application logs while maintaining scalability, security, and high performance.

Component	Technology Stack
Database Engine	PostgreSQL
ORM	SQLAlchemy
Migration Tool	Alembic

2. Database Architecture

The system architecture follows a layered flow from client requests down to persistent storage:

Client ──► FastAPI Backend ──► SQLAlchemy ORM ──► PostgreSQL Database

3. Tables Overview

The core schema consists of the following 11 entities:

- users - Stores core user accounts and profiles.
- user_sessions - Manages active JWT authentication sessions.
- ai_conversations - Stores prompt history and response metrics.
- wallet_scans - Stores risk evaluation results for wallet addresses.
- token_scans - Tracks token analysis and honeypot detection.
- contract_scans - Details smart contract security verification.
- website_scans - Evaluates domain and web security scores.
- transaction_scans - Analyzes specific blockchain transactions.

- scan_history - Provides a unified log of all scan types.
- notifications - Stores user alerts and system notifications.
- audit_logs - Records audit trails and system activity logs.

4. Detailed Table Specifications

4.1. users

Purpose: Store user accounts and identity information.

Field Name	Type / Description
id	UUID Primary Key
name	Full Name
email	Unique Email Address
password_hash	Bcrypt Hashed Password
profile_image	Image URL / Path
role	User Role (e.g., User, Admin)
status	Account Status (Active, Suspended)
created_at	Timestamp
updated_at	Timestamp

4.2. user_sessions

Purpose: Store active user login sessions.

Field Name	Description
id	Session ID
user_id	Foreign Key -> users.id

Field Name	Description
jwt_token	JWT Token string
device	Device / User Agent details
ip_address	Client IP address
expires_at	Expiration Timestamp
created_at	Creation Timestamp

4.3. ai_conversations

Purpose: Store AI assistant chat interactions and metadata.

Field Name	Description
id	Conversation ID
user_id	Foreign Key -> users.id
prompt	User input prompt text
response	AI generated response
tokens_used	Token usage count
model	Model identifier name
created_at	Timestamp

4.4. wallet_scans

Purpose: Store wallet risk analysis and assessment results.

Field Name	Description
id	Scan ID
user_id	Foreign Key -> users.id
wallet_address	Blockchain wallet address
risk_score	Numerical risk score
risk_level	Categorical risk rating (Low, Medium, High)
summary	Summary narrative
recommendation	Actionable security guidance
created_at	Timestamp

4.5. token_scans

Purpose: Store crypto token scans, liquidity metrics, and honeypot detection.

Field Name	Description
id	Scan ID
user_id	Foreign Key -> users.id
contract_address	Token contract address
token_name	Token name
symbol	Token ticker symbol
risk_score	Calculated risk score
risk_level	Risk classification

Field Name	Description
liquidity	Liquidity pool information
honeypot	Boolean / Status indicating honeypot risk
recommendation	System recommendation
created_at	Timestamp

4.6. contract_scans

Purpose: Store deep smart contract security analyses.

Field Name	Description
id	Scan ID
user_id	Foreign Key -> users.id
contract_address	Smart contract address
verified	Boolean source verification status
proxy_contract	Boolean / Proxy status indicator
risk_score	Calculated risk score
risk_level	Risk classification
recommendation	Actionable findings
created_at	Timestamp

4.7. website_scans

Purpose: Store Web3 / dApp URL and SSL safety scans.

Field Name	Description
id	Scan ID
user_id	Foreign Key -> users.id
website_url	Target Web URL
ssl_status	SSL Certificate status
domain_age	Domain registration age
trust_score	Trust rating score
risk_level	Assessed risk level
recommendation	Action recommendation
created_at	Timestamp

4.8. transaction_scans

Purpose: Store analysis of specific on-chain transaction hashes.

Field Name	Description
id	Scan ID
user_id	Foreign Key -> users.id
transaction_hash	Transaction Hash identifier
network	Blockchain network name
risk_score	Calculated risk score
risk_level	Assessed risk level

Field Name	Description
summary	Analysis summary
recommendation	Security guidance
created_at	Timestamp

4.9. scan_history

Purpose: Centralized index referencing all performed scan types for quick queries.

Field Name	Description
id	History Record ID
user_id	Foreign Key -> users.id
scan_type	Type of scan (Wallet, Token, Contract, Website, Transaction)
reference_id	ID of specific scan entry in respective scan table
risk_score	Overall risk score
created_at	Timestamp

4.10. notifications

Purpose: Store alerts and user system notifications.

Field Name	Description
id	Notification ID
user_id	Foreign Key -> users.id
title	Notification Header

Field Name	Description
message	Notification Body text
type	Alert Type (e.g., Warning, Info, Critical)
is_read	Boolean status
created_at	Timestamp

4.11. audit_logs

Purpose: Track security-critical system actions and administrative events.

Field Name	Description
id	Log Entry ID
user_id	Foreign Key -> users.id
action	Action identifier (e.g., Login, PasswordChange)
resource	Target Resource
status	Execution Status (Success, Failed)
ip_address	Originating IP address
created_at	Timestamp

5. Database Relationships

The central entry point of the relational hierarchy is the users table:

- **1:N Relationships:** Each user can have multiple active sessions, conversations, scan history items, specific scan outputs, notifications, and audit logs.
- **Referential Integrity:** Foreign keys enforce constraints across all dependent records linked to users.id.

6. Database Indexes & Query Optimization

To maintain low query latency and support fast lookups under heavy read traffic, indexes are established on the following fields:

Indexed Column	Primary Query Benefit
email	Fast user lookup during login / authentication
wallet_address	Rapid querying of cached wallet security scans
contract_address	Quick retrieval of smart contract and token scan metrics
transaction_hash	Efficient transaction lookup by on-chain hash
website_url	Fast domain lookup for web safety ratings
user_id	Fast join execution across user activities and history
created_at	Efficient time-series filtering and sorting

7. Security Compliance & Implementation

- **Password Security:** Encrypt all passwords using bcrypt hashing algorithm. Plaintext storage is strictly prohibited.
- **Primary Key Strategy:** Utilize globally unique identifiers (UUIDs) across primary key fields to prevent enumeration attacks.
- **Transport Security:** Enable mandatory SSL/TLS database connections for all backend service communication.
- **Access Control:** Enforce strict principle-of-least-privilege database roles and permissions.
- **Injection Defense:** Mitigate SQL Injection risks completely through SQLAlchemy ORM parameter binding and query abstraction.

8. Backup & Disaster Recovery Strategy

- **Daily Automated Backups:** Incremental backups executed daily during off-peak hours.

- **Weekly Full Backups:** Complete baseline database backup performed weekly.
- **Point-in-Time Recovery (PITR):** Enabled via continuous Write-Ahead Log (WAL) archiving.
- **Encryption at Rest:** All backup artifacts are stored in encrypted object storage.

9. Future Database Expansion Framework

Planned modules and schema extensions for future platform iterations:

- wallet_monitoring - Real-time tracking and automated alerts for designated addresses.
- ai_memory - Vector embeddings and persistent contextual memory for AI agents.
- blockchain_events - Webhook triggers and real-time smart contract events.
- team_workspaces - Multi-tenant team roles and shared analytical dashboards.
- api_keys - Rate limits and API token authentication for third-party developers.
- billing & subscriptions - Payment gateway integrations and SaaS tiers.
- security_reports - Exportable PDF/HTML audit reports.

10. Expected Outcome

The designed database architecture provides a secure, scalable, and well-structured foundation for **Aegivex AI**. It ensures efficient storage and retrieval of user accounts, blockchain security analysis, AI conversations, scan histories, and notifications while supporting high performance required for production deployment during and after the OKX.AI Genesis Hackathon.